Decorator Design Pattern — Dynamic Behavior Composition without Inheritance Explosion

1 Problem Statement (motivating scenario)

Primary scenario: Notification service. You operate a shared `NotificationService` used by multiple clients. Initially, it only sends email:

- `notify(String text)` → deliver via Email.

Now, new requirements arrive *per client*:

- Client A: "Send **Email** + **WhatsApp**."
- Client B: "Send **Email** + **SMS**."
- Client C: "Send **Email** + **WhatsApp** + **Slack**."
- Others may ask for Telegram, Teams, rate limiting, retries, templating, audit logs, metrics, etc.

These behaviors are *optional*, *combinable*, and must be configurable per tenant or environment. Order may matter (e.g., apply formatting/logging before sending, or measure metrics around the full stack).

Secondary scenario: game development. A character or projectile acquires runtime "modifiers" (e.g., Poison, Fire, Armor Piercing, Critical Strike). Modifiers stack and order affects outcome (e.g., pierce → crit vs. crit → pierce).

2 Forces, Constraints, Goals

- **Runtime configuration.** Per-tenant policies and feature flags require toggling channels/behaviors *without* new builds.
- **Combinatorial requirements.** Many behaviors are optional *and* combinable; naive inheritance explodes.
- **Order sensitivity.** Logging, retries, rate limiting, and multi-channel sends have order effects.
- **Performance/testability.** Each layer adds latency and potential failure modes; layers must be measurable and testable in isolation.

3 Naïve Alternatives and Why They Fail (with SOLID references)

Solution 1 — Subclass-per-combination ("create more classes")

Idea. Keep a simple `Notifier` API but create one class per combination:

`EmailNotifier`, `EmailAndWhatsAppNotifier`, `EmailAndSmsNotifier`, `EmailAndWhatsAppAndSmsNotifier`, ...

Why teams start here. It feels explicit and "clean" for a few variants.

Why it breaks.

- **Combinatorial explosion:** with N channels you can have up to $2^N - 1$ combinations.
- **Rigid evolution:** adding *one* new channel (e.g., Telegram) forces many new classes and factory/DI updates.
- **Duplication & drift:** common behavior (formatting, retries, audit) gets copy-pasted across classes and diverges.
- **Testing burden:** near-duplicate tests proliferate; coverage becomes noisy rather than meaningful.
- **OCP pain:** the type lattice must be edited for new needs, rather than extended by composition.

Solution 2 — One configurable class with booleans and `if/else`

Idea. Centralize in one class with flags:

```java
// File: ConfigurableNotifier.java
class ConfigurableNotifier implements Notifier {
  private final EmailGateway email;
  private final WhatsAppGateway wa; // may be null
  private final SmsGateway sms;     // may be null
  private final boolean enableWhatsApp, enableSms;

  public void notify(String text) {
    email.send(text);
    if (enableWhatsApp) wa.send(text);
    if (enableSms) sms.send(text);
    // later: if (enableSlack) ...
  }
}
```

Why it's tempting. One place to read; fast to add "just one more if".

Why not to prefer it.

- **SRP violation:** class decides *which* channels, *how* to send, and where to log/retry — too many reasons to change.
- **OCP violation:** each new channel or rule edits `notify`.
- **Constructor bloat:** growing parameter list and nulls for unused dependencies.
- **Branch thickets:** business rules accrete into brittle, interdependent conditionals.
- **Testing matrix:** many flag combinations to cover; rare paths rot.
- **Hidden coupling:** cross-cutting concerns (metrics, rate limiting, retries) leak into the same method.

4 Decorator: Core Idea and Intuition

Definition. Keep a stable component interface (e.g., `Notifier#notify`). Implement the baseline capability as a concrete component (`EmailNotifier`). Add optional behaviors as *decorators* that:

1. implement the same interface,
2. wrap another component,
3. forward the call and add behavior before/after.

Intuition. Like wrapping a parcel with optional services (fragile, insurance, tracking) or an onion: stack layers, remove layers, keep the core unchanged. Each wrapper is still a `Notifier`, so clients remain ignorant of how many layers exist.

5 Design Heuristics

- Keep the **component interface minimal** so decorators remain simple.
- **Document ordering** when it affects semantics (e.g., metrics outermost).
- Prefer **stateless** decorators; if stateful (e.g., cache/rate limit), keep state encapsulated and avoid globals.
- Expose **configuration** via constructors/builders instead of runtime `instanceof`.
- Preserve contracts (LSP): a decorator must not change expected semantics of the interface.

6 Detailed Java Example (HTTP client pipeline)

Although our motivating story is notifications, HTTP pipelines are a canonical playground for Decorator: retries, caching, compression, auth, metrics. Below is a cohesive, compilable set; each block can be its own file.

Component and DTOs

```java
// File: HttpRequest.java
import java.util.*;

public class HttpRequest {
  public final String method;
  public final String url;
  public final Map<String, String> headers;
  public final byte[] body;

  public HttpRequest(String method, String url, Map<String,String> headers,
      byte[] body) {
    this.method = method;
    this.url = url;
    this.headers = headers != null ? new LinkedHashMap<>(headers) : new
       LinkedHashMap<>();
    this.body = body != null ? body.clone() : null;
  }

  public HttpRequest withHeader(String k, String v) {
    Map<String,String> h = new LinkedHashMap<>(this.headers);
```

```
    h.put(k, v);
    return new HttpRequest(this.method, this.url, h, this.body);
  }
}
```

```
// File: HttpResponse.java
import java.util.*;

public class HttpResponse {
  public final int status;
  public final Map<String, String> headers;
  public final byte[] body;

  public HttpResponse(int status, Map<String,String> headers, byte[] body)
     {
    this.status = status;
    this.headers = headers != null ? new LinkedHashMap<>(headers) : new
       LinkedHashMap<>();
    this.body = body != null ? body.clone() : null;
  }
}
```

```
// File: HttpClient.java
public interface HttpClient {
  HttpResponse send(HttpRequest req);
}
```

Concrete component

```
// File: BaseHttpClient.java
import java.nio.charset.StandardCharsets;
import java.util.*;

public class BaseHttpClient implements HttpClient {
  @Override
  public HttpResponse send(HttpRequest req) {
    String echo = "BaseHttpClient -> " + req.method + " " + req.url;
    return new HttpResponse(200, Map.of("X-Echo", "ok"), echo.getBytes(
       StandardCharsets.UTF_8));
  }
}
```

Decorator base

```
// File: HttpClientDecorator.java
public abstract class HttpClientDecorator implements HttpClient {
  protected final HttpClient inner;
  protected HttpClientDecorator(HttpClient inner) { this.inner = inner; }
  @Override public HttpResponse send(HttpRequest req) {
    return inner.send(req);
  }
}
```

Concrete decorators

Retries with backoff.

```java
// File: RetryingHttpClient.java
public class RetryingHttpClient extends HttpClientDecorator {
  private final int maxAttempts;
  private final long baseBackoffMillis;

  public RetryingHttpClient(HttpClient inner, int maxAttempts, long
      baseBackoffMillis) {
    super(inner);
    this.maxAttempts = Math.max(1, maxAttempts);
    this.baseBackoffMillis = Math.max(0, baseBackoffMillis);
  }

  @Override
  public HttpResponse send(HttpRequest req) {
    int attempt = 0;
    HttpResponse last = null;
    while (attempt < maxAttempts) {
      attempt++;
      last = inner.send(req);
      if (last.status < 500) return last;
      try { Thread.sleep(baseBackoffMillis * attempt); } catch (
          InterruptedException ie) { Thread.currentThread().interrupt();
          break; }
    }
    return last;
  }
}
```

Simple caching with TTL.

```java
// File: CachingHttpClient.java
import java.util.*;
import java.util.concurrent.*;

public class CachingHttpClient extends HttpClientDecorator {
  private static final class Entry {
    final HttpResponse resp; final long expiry;
    Entry(HttpResponse r, long e) { resp = r; expiry = e; }
  }
  private final ConcurrentMap<String, Entry> cache = new ConcurrentHashMap
      <>();
  private final long ttlMillis;

  public CachingHttpClient(HttpClient inner, long ttlMillis) {
    super(inner);
    this.ttlMillis = Math.max(0, ttlMillis);
  }

  private String key(HttpRequest r) {
    return r.method + " " + r.url;
  }

  @Override
  public HttpResponse send(HttpRequest req) {
    if ("GET".equalsIgnoreCase(req.method)) {
      String k = key(req);
```

```
      Entry e = cache.get(k);
      long now = System.currentTimeMillis();
      if (e != null && e.expiry > now) return e.resp;
      HttpResponse fresh = inner.send(req);
      cache.put(k, new Entry(fresh, now + ttlMillis));
      return fresh;
    }
    return inner.send(req);
  }
}
```

Compression negotiation.

```
// File: CompressingHttpClient.java
import java.util.*;

public class CompressingHttpClient extends HttpClientDecorator {
  public CompressingHttpClient(HttpClient inner) { super(inner); }
  @Override
  public HttpResponse send(HttpRequest req) {
    HttpRequest with = req.withHeader("Accept-Encoding", "gzip");
    return inner.send(with);
  }
}
```

Authentication header.

```
// File: AuthHttpClient.java
import java.util.function.Supplier;

public class AuthHttpClient extends HttpClientDecorator {
  private final Supplier<String> tokenSupplier;
  public AuthHttpClient(HttpClient inner, Supplier<String> tokenSupplier) {
    super(inner);
    this.tokenSupplier = tokenSupplier;
  }
  @Override
  public HttpResponse send(HttpRequest req) {
    String token = tokenSupplier.get();
    HttpRequest with = req.withHeader("Authorization", "Bearer " + token);
    return inner.send(with);
  }
}
```

Metrics (timing + counting).

```
// File: MetricsHttpClient.java
import java.util.concurrent.atomic.AtomicLong;

public class MetricsHttpClient extends HttpClientDecorator {
  private final AtomicLong calls = new AtomicLong();
  private final AtomicLong totalNanos = new AtomicLong();

  public MetricsHttpClient(HttpClient inner) { super(inner); }

  @Override
  public HttpResponse send(HttpRequest req) {
    long t0 = System.nanoTime();
    try { return inner.send(req); }
```

```
    finally {
      long dur = System.nanoTime() - t0;
      calls.incrementAndGet();
      totalNanos.addAndGet(dur);
      System.out.println("[metrics] call=" + calls.get() + " avg_ms=" + (
          totalNanos.get()/1_000_000.0 / calls.get()));
    }
  }
}
```

Wiring examples and order effects

```
// File: HttpClientWiringDemo.java
import java.util.Map;

public class HttpClientWiringDemo {
  public static void main(String[] args) {
    HttpClient base = new BaseHttpClient();
    HttpRequest r = new HttpRequest("GET", "https://api.example.com/data",
        Map.of(), null);

    // Order A: Metrics outside; Retry outside Cache.
    HttpClient a = new MetricsHttpClient(
                      new RetryingHttpClient(
                        new CachingHttpClient(
                          new AuthHttpClient(
                            new CompressingHttpClient(base),
                            () -> "token-abc"), 5_000),
                        3, 50));
    System.out.println("=== ORDER A ===");
    a.send(r); a.send(r); // second call should hit cache

    // Order B: Cache outside Retry; different semantics.
    HttpClient b = new MetricsHttpClient(
                      new CachingHttpClient(
                        new RetryingHttpClient(
                          new AuthHttpClient(base, () -> "token-xyz"), 3,
                              50),
                        5_000));
    System.out.println("=== ORDER B ===");
    b.send(r); b.send(r);
  }
}
```

7 Secondary Java Example (game dev)

```
// File: DamageSource.java
public interface DamageSource {
  double applyTo(double baseHp);
  String describe();
}
```

```
// File: BaseHit.java
public class BaseHit implements DamageSource {
```

```java
  private final double damage;
  public BaseHit(double damage) { this.damage = damage; }
  @Override public double applyTo(double baseHp) { return Math.max(0,
      baseHp - damage); }
  @Override public String describe() { return "Base(" + damage + ")"; }
}
```

```java
// File: DamageDecorator.java
public abstract class DamageDecorator implements DamageSource {
  protected final DamageSource inner;
  protected DamageDecorator(DamageSource inner) { this.inner = inner; }
  @Override public double applyTo(double baseHp) { return inner.applyTo(
      baseHp); }
  @Override public String describe() { return inner.describe(); }
}
```

```java
// File: ArmorPiercing.java
public class ArmorPiercing extends DamageDecorator {
  private final double flatReduction;
  public ArmorPiercing(DamageSource inner, double flatReduction) {
    super(inner); this.flatReduction = flatReduction;
  }
  @Override public double applyTo(double baseHp) {
    double boosted = Math.max(0, baseHp - flatReduction);
    return inner.applyTo(boosted);
  }
  @Override public String describe() { return inner.describe() + " -> AP("
      + flatReduction + ")"; }
}
```

```java
// File: CriticalStrike.java
public class CriticalStrike extends DamageDecorator {
  private final double multiplier;
  public CriticalStrike(DamageSource inner, double multiplier) {
    super(inner); this.multiplier = multiplier;
  }
  @Override public double applyTo(double baseHp) {
    double hpAfter = inner.applyTo(baseHp);
    double delta = baseHp - hpAfter;
    double critDelta = delta * multiplier;
    return Math.max(0, baseHp - critDelta);
  }
  @Override public String describe() { return inner.describe() + " -> Crit(
      x" + multiplier + ")"; }
}
```

```java
// File: PoisonDamage.java
public class PoisonDamage extends DamageDecorator {
  private final double dot;
  public PoisonDamage(DamageSource inner, double dot) { super(inner); this.
      dot = dot; }
  @Override public double applyTo(double baseHp) {
    double hpAfter = inner.applyTo(baseHp);
    return Math.max(0, hpAfter - dot);
  }
  @Override public String describe() { return inner.describe() + " ->
      Poison(" + dot + ")"; }
```

```
}
```

```
// File: GameDemo.java
public class GameDemo {
  public static void main(String[] args) {
    double hp = 150;
    DamageSource base = new BaseHit(40);

    DamageSource buildA = new PoisonDamage(new CriticalStrike(new
        ArmorPiercing(base, 10), 1.5), 8);
    System.out.println(buildA.describe() + " => HP " + hp + " -> " + buildA
        .applyTo(hp));

    DamageSource buildB = new CriticalStrike(new PoisonDamage(new
        ArmorPiercing(base, 10), 8), 1.5);
    System.out.println(buildB.describe() + " => HP " + hp + " -> " + buildB
        .applyTo(hp));
  }
}
```

8 Valid doubts

Valid doubt: "Isn't this just Proxy?"
Answer: Proxy focuses on access/indirection (remote, virtual, protection). Decorator preserves the same interface but *adds responsibilities* and supports stacking.

Valid doubt: "Why not AOP or a framework pipeline?"
Answer: AOP weaves concerns at build/run time and can obscure order; decorators are explicit, library-level, and unit-testable with precise composition.

Valid doubt: "Can decorators change return types or throw new exceptions?"
Answer: They must honor the interface contract (LSP). Surface errors consistently; do not widen/narrow types beyond the abstraction.

9 Testing Strategy

Test each decorator in isolation, then verify order-sensitive compositions and overall contracts. Include micro-benchmarks for per-layer latency.
JUnit sketch (retry).

```
// File: RetryingHttpClientTest.java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class RetryingHttpClientTest {
  static class FlakyClient implements HttpClient {
    int calls = 0;
    @Override public HttpResponse send(HttpRequest req) {
      calls++;
      if (calls < 3) return new HttpResponse(503, java.util.Map.of(), null)
          ;
      return new HttpResponse(200, java.util.Map.of(), null);
    }
  }

  @Test
```

```
  void retries_until_success() {
    FlakyClient flaky = new FlakyClient();
    HttpClient client = new RetryingHttpClient(flaky, 5, 0);
    HttpResponse r = client.send(new HttpRequest("GET","u",java.util.Map.of
        (),null));
    assertEquals(200, r.status);
    assertEquals(3, flaky.calls);
  }
}
```

10 Mapping to SOLID

SRP. One responsibility per decorator (SMS send, WhatsApp send, retry, metrics).
OCP. Add new behaviors by adding classes; no edits to existing ones.
LSP. Decorators remain substitutable for the component.
ISP. Keep the component interface lean.
DIP. Clients depend on abstractions (`Notifier`, `HttpClient`), not concrete stacks.

11 Pitfalls and How to Avoid Them

- **Order confusion:** publish recommended orders (e.g., metrics outermost; auth before network call).
- **State leakage:** prefer immutable configs and request-scoped state; avoid globals.
- **Swallowed errors:** log at boundaries and rethrow; avoid double-wrapping exceptions.
- **Over-decoration:** measure latency and prune low-value layers.
- **Type checks in decorators:** replace with constructor-injected collaborators.

12 When to Use / When Not to Use

Use when behaviors are cross-cutting, optional, combinable, and order-sensitive around a stable interface (like our notification service). **Avoid** when one algorithm choice suffices (Strategy), when adapting interfaces (Adapter), or mediating access (Proxy).

13 Mini Walkthrough / Exercise

Start with `EmailNotifier`. Add a `WhatsAppDecorator` and `SmsDecorator` as separate classes that implement `Notifier` and wrap another `Notifier`. Compose per client:

- Client A: `new WhatsAppDecorator(new EmailNotifier(...))`.
- Client B: `new SmsDecorator(new EmailNotifier(...))`.
- Client C: `new SmsDecorator(new WhatsAppDecorator(new EmailNotifier(...)))`.

Add `MetricsDecorator` around the final stack to measure end-to-end latency. Remove WhatsApp by *recomposing* without its wrapper.

14 Takeaway

Decorator prevents inheritance explosion and tangled conditionals by turning optional, cross-cutting behaviors into *small, composable layers* around a stable interface. You gain runtime flexibility, clear ordering, and modular testability — exactly what evolving notification systems (and many other pipelines) need.